Software Requirements Specification For

Smart Expense Tracker Web System

Version 1.0

Prepared by M.C. Nuwan RAT/IT/2324/F/0055

Under the Supervision of Mr. Thilina Bandara

Department of Information Technology
Advanced Technological Institute Rathnapura

May 2026

Table of Contents

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a detailed description of the Smart Expense Tracker Web System. This document specifies the functional and non-functional requirements, system constraints, and external interfaces for the development of the web-based expense tracking application. It serves as a baseline for design, development, testing, and validation activities.

The Smart Expense Tracker Web System is designed to help individuals efficiently manage and monitor their personal finances by automating income and expense tracking, providing visual analytics, and enabling budget control with automated alerts.

1.2 Document Conventions

This document follows the IEEE 830-1998 standard format for Software Requirements Specifications. The following conventions are used throughout this document:

- Requirement identifiers follow the format R-[SUBSYSTEM]-[NUMBER] (e.g., RAUTH-1).
- 'Shall' indicates mandatory requirements.
- 'Should' indicates recommended requirements.
- 'May' indicates optional requirements.
- Priority levels: High (essential), Medium (important), Low (desirable).
- All monetary values are referenced in the user's local currency.

1.3 Intended Audience and Reading Suggestions

This SRS document is intended for the following stakeholders:

- Software Engineers and Developers: Responsible for designing and implementing the system. Should read the entire document, with particular attention to Sections 3 and 4.
- Project Supervisor and Academic Evaluators: Responsible for overseeing the project and evaluating deliverables. Should review Sections 1, 2, and 5 for project alignment. • Quality Assurance Team: Responsible for testing and validation. Should focus on Section 3 (System Features) and Section 5 (Nonfunctional Requirements).
- End Users: Individuals who will use the system for personal finance management. May refer to Section 3.1 and Section 4.1 for user-facing features and interface descriptions.

It is recommended to read this document sequentially, referencing the Glossary (Appendix A) for unfamiliar terms.

1.4 Project Scope

The Smart Expense Tracker Web System is a web-based application designed for personal financial management. The system enables users to:

- Register and authenticate securely to access personal financial data.
- Record, categorize, and manage income and expense transactions.
- View financial summaries and visualize spending patterns through charts and graphs.
- Set monthly budget limits and receive automated alerts when approaching or exceeding limits.
- Generate financial reports for better decision-making.

The system is developed as a university project (HNDIT4052) under the supervision of Mr. Thilina Bandara at the Advanced Technological Institute, Rathnapura. The project utilizes React.js for the frontend, Node.js (Express) for the backend, and MongoDB for data storage.

The scope explicitly excludes:

- Multi-currency support (Phase 1).
- Integration with banking APIs or external financial institutions.
- Tax calculation and filing features.
- Mobile native application development (responsive web design only).

1.5 References

The following documents and standards are referenced in this SRS:

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
- React.js Documentation, Meta Platforms, Inc., https://react.dev/
- Node.js Documentation, OpenJS Foundation, https://nodejs.org/
- MongoDB Documentation, MongoDB, Inc., https://www.mongodb.com/docs/ • Express.js Documentation, OpenJS Foundation, https://expressjs.com/
- JSON Web Token (JWT) Standard, RFC 7519, IETF.

2. Overall Description

2.1 Product Perspective

The Smart Expense Tracker Web System is a standalone, self-contained web application designed for personal financial management. It operates independently without requiring integration with external financial systems or banking APIs.

The system architecture follows a client-server model:

- Client Tier: React.js-based Single Page Application (SPA) running in modern web browsers.
- Application Tier: Node.js with Express.js framework handling business logic and API endpoints.
- Data Tier: MongoDB NoSQL database for persistent storage of user data, transactions, and categories.

The system is designed to be accessed via internet-connected devices with standard web browsers. All financial data is user-specific and isolated through authentication and authorization mechanisms.

2.2 Product Features

The Smart Expense Tracker Web System comprises the following major feature categories:

1. User Authentication System: Secure user registration, login, logout, and profile management using JWT-based authentication.

2. Income Management System: Record, edit, delete, and view income transactions with categorization (e.g., Salary, Bonus, Freelance, Investment).

3. Expense Management System: Record, edit, delete, and view expense transactions with categorization (e.g., Food, Transport, Bills, Entertainment, Healthcare).

4. Category Management System: Create, update, delete custom categories for both income and expenses.

5. Dashboard and Visualization System: Display financial summaries, pie charts for expense distribution, bar charts for income vs. expense trends, and balance calculations.

6. Budget Management and Alert System: Set monthly budget limits per category, track spending against budgets, and generate visual and email alerts when thresholds are approached or exceeded.

2.3 User Classes and Characteristics

The system identifies the following user classes:

• Standard User (End User):
- Primary user of the system for personal finance management.

- Has full CRUD (Create, Read, Update, Delete) permissions on their own transactions and categories.
- Can view dashboards, set budgets, and receive alerts.
- Technical expertise: Basic to intermediate computer literacy; familiar with web applications.
- Educational level: General public with basic financial literacy.

• System Administrator (Developer/Maintenance Role):
- Responsible for system maintenance, user management, and database administration.
- Has access to system logs and user statistics (anonymized).
- Can manage system-wide configurations and perform backups.
- Technical expertise: Advanced; proficient in web technologies and database management.

• Guest User (Unauthenticated Visitor):
- Can view a public landing page with system features overview.
- Cannot access financial data or dashboard features.
- Must register to access full system functionality.

2.4 Operating Environment

The Smart Expense Tracker Web System operates in the following environment:

Client Environment:
• Modern web browsers: Google Chrome (v90+), Mozilla Firefox (v88+), Microsoft Edge (v90+), Safari (v14+).
• Minimum screen resolution: 768px width (responsive design supports mobile, tablet, and desktop).
• Internet connection required for all operations.

Server Environment:
• Operating System: Linux (Ubuntu 20.04 LTS or compatible) or Windows Server 2019.
• Runtime Environment: Node.js v18.x LTS.
• Web Server: Express.js framework running on HTTP/HTTPS.
• Database: MongoDB v6.0+ (local or cloud-hosted via MongoDB Atlas).
• Memory: Minimum 2GB RAM (4GB recommended).
• Storage: Minimum 10GB available space for application and database files.

Development Environment:
• IDE: Visual Studio Code.
• Version Control: Git (GitHub repository).
• Package Manager: npm (Node Package Manager).

2.5 Design and Implementation Constraints

The following constraints apply to the design and implementation of the system:

- Technology Stack Constraint: The system must be developed using React.js (frontend), Node.js with Express (backend), and MongoDB (database) as specified in the project proposal.

- Authentication Constraint: User authentication must implement JWT (JSON Web Tokens) for stateless session management.

- Budget Constraint: As a university project, development is limited to a 7-week timeline with limited resources for cloud hosting or third-party services.

- Browser Compatibility: The system must function correctly on the browsers specified in Section 2.4.

- Data Privacy: Financial data must be stored securely; plaintext passwords are prohibited. All passwords must be hashed using bcrypt or equivalent algorithm.

- Responsive Design: The UI must adapt to mobile, tablet, and desktop viewports without separate codebases.

- Open Source Dependencies: Where possible, open-source libraries should be used to minimize licensing costs.

- Academic Integrity: The system must be developed as original work for academic evaluation; plagiarism of existing commercial solutions is prohibited.

2.6 User Documentation

The following user documentation will be delivered with the system:

- User Guide: A comprehensive guide covering registration, login, adding transactions, setting budgets, and interpreting dashboard visualizations. Available as an online HTML help page and downloadable PDF.

- Installation and Deployment Guide: Technical documentation for deploying the system on a server, including environment setup, database configuration, and security settings.

- API Documentation: Auto-generated documentation (using Swagger/OpenAPI) describing all backend endpoints, request/response formats, and authentication requirements.

- README File: Project overview, setup instructions, and contribution guidelines for open-source collaboration, hosted in the GitHub repository.

- In-Application Help: Contextual help tooltips and guided tours for first-time users within the web interface.

2.7 Assumptions and Dependencies

The following assumptions and dependencies affect the requirements stated in this SRS:

Assumptions:

- Users have access to a stable internet connection and a modern web browser.
- Users possess basic financial literacy to categorize transactions and interpret financial summaries.
- The development team has access to development machines with Node.js, MongoDB, and Git installed.
- MongoDB Atlas or a local MongoDB instance will be available for database operations.
- The project supervisor will provide timely feedback on deliverables.

Dependencies:

- React.js ecosystem dependencies (react-router-dom, axios, recharts, etc.).
- Node.js dependencies (express, mongoose, jsonwebtoken, bcryptjs, cors, dotenv, etc.).
- MongoDB database service availability.
- npm registry accessibility for package installation.
- GitHub repository for version control and collaboration.

If any of these dependencies become unavailable, appropriate alternatives must be identified, and the SRS may require revision.

3. System Features

3.1 User Authentication System

3.1.1 Description and Priority

This feature enables users to register for a new account, log in securely, and manage their profile. Authentication is the gateway to all other system features and is therefore

classified as High Priority. Failure to provide secure authentication will compromise all user financial data and system integrity.

The authentication mechanism uses JWT (JSON Web Tokens) to maintain secure, stateless sessions between the client and server.

3.1.2 Stimulus/Response Sequences

Registration Sequence:

1. User navigates to the registration page.
2. User enters full name, email address, password, and confirms password.
3. System validates input format (email syntax, password strength, matching passwords).
4. System checks if email already exists in the database.
5. If valid and unique, system hashes password using bcrypt and stores user record in MongoDB.
6. System generates JWT token and redirects user to the dashboard.
7. If validation fails, system displays appropriate error messages.

Login Sequence:

1. User navigates to the login page.
2. User enters email address and password.
3. System retrieves user record by email.
4. System compares provided password with stored hash.
5. If valid, system generates JWT token and redirects to dashboard.
6. If invalid, system displays 'Invalid credentials' message and increments failed attempt counter.
7. After 5 consecutive failed attempts, system temporarily locks account for 15 minutes.

Logout Sequence:

1. User clicks logout button.
2. System invalidates JWT token on client side.
3. System redirects user to the login page.

3.1.3 Functional Requirements

R-AUTH-1: The system shall allow users to register with a unique email address, full name, and password.

R-AUTH-2: The system shall enforce password complexity: minimum 8 characters, at least one uppercase letter, one lowercase letter, one number, and one special character. R-AUTH-3: The system shall validate email format using standard regex patterns. RAUTH-4: The system shall hash all passwords using bcrypt with a minimum salt round of 10.

R-AUTH-5: The system shall generate a JWT token upon successful authentication, valid for 24 hours.

R-AUTH-6: The system shall verify JWT token on every protected API request.
RAUTH-7: The system shall allow users to update their profile information (name, email, password).
R-AUTH-8: The system shall implement account lockout after 5 consecutive failed login attempts for 15 minutes.
R-AUTH-9: The system shall provide a 'Forgot Password' feature allowing password reset via email verification.
R-AUTH-10: The system shall maintain an audit log of all authentication events (login, logout, failed attempts, password changes).

3.2 Income Management System

3.2.1 Description and Priority

This feature allows users to record, manage, and categorize all income transactions. Users can add income sources such as salary, bonuses, freelance payments, investments, and other earnings. This feature is classified as High Priority as it is fundamental to the system's core purpose of financial tracking.

3.2.2 Stimulus/Response Sequences Add

Income Sequence:
1. User navigates to the 'Add Income' page from the dashboard.
2. User selects income category from predefined or custom list.
3. User enters amount, date, description, and optional notes.
4. User clicks 'Save' button.
5. System validates amount (positive number), date (not future date), and required fields.
6. System stores transaction in MongoDB with user ID reference.
7. System updates dashboard totals and budget calculations.
8. System displays success message and redirects to income list.

Edit Income Sequence:
1. User navigates to income list and clicks 'Edit' on a specific transaction.
2. System pre-populates form with existing data.
3. User modifies fields and clicks 'Update'.
4. System validates changes and updates the record.
5. System recalculates affected summaries and budgets.

Delete Income Sequence:
1. User clicks 'Delete' on an income transaction.
2. System displays confirmation dialog.
3. Upon confirmation, system marks record as deleted (soft delete) or removes it.
4. System updates all dependent calculations.

3.2.3 Functional Requirements

R-INC-1: The system shall allow users to add income transactions with the following fields: amount (required, positive decimal), category (required), date (required, not future date), description (optional, max 255 chars), notes (optional, max 1000 chars). R-INC-2: The system shall provide default income categories: Salary, Bonus, Freelance, Investment, Gift, Other.
R-INC-3: The system shall allow users to create, edit, and delete custom income categories.
R-INC-4: The system shall display a chronological list of all income transactions with sorting (date, amount, category) and filtering options.
R-INC-5: The system shall calculate and display total income for selectable date ranges (day, week, month, year, custom).
R-INC-6: The system shall allow editing of existing income transactions with validation.
R-INC-7: The system shall support soft delete (mark as deleted) with optional hard delete confirmation.
R-INC-8: The system shall prevent duplicate entries within a 5-minute window for identical amounts and categories.
R-INC-9: The system shall export income data to CSV format.
R-INC-10: The system shall display income trends in bar chart format on the dashboard.

3.3 Expense Management System

3.3.1 Description and Priority

This feature allows users to record, manage, and categorize all expense transactions. Expenses can be classified into categories such as food, transport, bills, entertainment, healthcare, education, and others. This feature is classified as High Priority as expense tracking is the primary function of the system.

3.3.2 Stimulus/Response Sequences Add

Expense Sequence:

1. User navigates to 'Add Expense' page.
2. User selects expense category from predefined or custom list.
3. User enters amount, date, description, payment method (cash/card/digital), and optional receipt image.
4. User clicks 'Save'.
5. System validates amount (positive number), date, and required fields.
6. System stores transaction in MongoDB.
7. System checks budget limits and triggers alerts if necessary.
8. System updates dashboard and redirects to expense list.

Bulk Expense Entry Sequence:

1. User selects 'Bulk Entry' option.

2. User uploads CSV file with multiple expense records.
3. System parses and validates each record.
4. System reports validation errors (if any) and allows correction.
5. Upon validation, system imports all valid records.
6. System generates import summary report.

3.3.3 Functional Requirements

R-EXP-1: The system shall allow users to add expense transactions with: amount (required, positive decimal), category (required), date (required, not future date), description (optional), payment method (optional), receipt image upload (optional, max 5MB, formats: JPG, PNG, PDF).
R-EXP-2: The system shall provide default expense categories: Food, Transport, Bills, Entertainment, Healthcare, Education, Shopping, Rent, Other.
R-EXP-3: The system shall allow custom expense category creation, editing, and deletion.
R-EXP-4: The system shall display expenses in list view with sort (date, amount, category) and filter options.
R-EXP-5: The system shall calculate total expenses for selectable date ranges. R-EXP-6: The system shall support recurring expense setup (daily, weekly, monthly, yearly) with automatic generation.
R-EXP-7: The system shall allow editing and soft deletion of expense records.
R-EXP-8: The system shall detect potential duplicate expenses and warn the user.
R-EXP-9: The system shall support bulk import of expenses via CSV template.
R-EXP-10: The system shall allow attachment of receipt images to expense records.
R-EXP-11: The system shall export expense data to CSV and PDF formats.

3.4 Category Management System

3.4.1 Description and Priority

This feature enables users to create, customize, and manage categories for both income and expenses. Proper categorization is essential for meaningful financial analysis and visualization. This feature is classified as Medium Priority.

3.4.2 Stimulus/Response Sequences

Create Category Sequence:
1. User navigates to 'Categories' section.
2. User clicks 'Add New Category'.
3. User enters category name, selects type (income/expense), chooses color/icon.
4. System validates name uniqueness within type.
5. System stores new category associated with user ID.
6. Category becomes available in transaction forms.

Delete Category Sequence:
1. User selects category to delete.
2. System checks if category has associated transactions.
3. If transactions exist, system prompts for re-categorization or prevents deletion.
4. If no transactions, system removes category after confirmation.

3.4.3 Functional Requirements

R-CAT-1: The system shall allow users to create custom categories with: name (required, unique within type, max 50 chars), type (income or expense), color (hex code for chart visualization), icon (from predefined set).
R-CAT-2: The system shall enforce unique category names within each type per user.
RCAT-3: The system shall prevent deletion of categories that have associated transactions; instead, offer re-categorization or archiving.
R-CAT-4: The system shall allow editing of custom category properties (name, color, icon).
R-CAT-5: The system shall not allow modification or deletion of default system categories.
R-CAT-6: The system shall display category usage statistics (number of transactions, total amount).
R-CAT-7: The system shall allow categories to be marked as 'active' or 'inactive' (hidden from selection).

3.5 Dashboard and Visualization System

3.5.1 Description and Priority

This feature provides users with a comprehensive financial overview through an interactive dashboard. It displays key financial metrics, charts, and summaries to help users understand their financial health. This feature is classified as High Priority as it delivers the primary value proposition of the system.

3.5.2 Stimulus/Response Sequences

Dashboard Load Sequence:
1. User successfully logs in and is redirected to the dashboard.
2. System queries database for user's financial data within the default date range (current month).
3. System calculates: total income, total expenses, current balance, savings rate.
4. System generates chart data: pie chart (expense by category), bar chart (income vs expense trend).
5. System renders dashboard components with data.
6. User can interact with date range selector to update all metrics dynamically.

Chart Interaction Sequence:
1. User hovers over pie chart segment.

2. System displays tooltip with category name, amount, and percentage of total.
3. User clicks on segment.
4. System filters transaction list to show only that category's transactions.
5. User clicks 'Reset' to return to full view.

3.5.3 Functional Requirements

R-DASH-1: The system shall display a summary card showing: total income, total expenses, net balance, and savings rate (percentage) for the selected period.
R-DASH-2: The system shall render a pie chart showing expense distribution by category with interactive tooltips.
R-DASH-3: The system shall render a bar chart comparing income vs. expenses over time (daily, weekly, monthly).
R-DASH-4: The system shall provide a date range selector with presets: Today, This Week, This Month, This Year, Last 30 Days, Custom Range.
R-DASH-5: The system shall display a recent transactions list (last 10 entries) with quick-view details.
R-DASH-6: The system shall show budget utilization progress bars for categories with active budgets.
R-DASH-7: The system shall highlight categories approaching or exceeding budget limits with color coding (green, yellow, red).
R-DASH-8: The system shall display a line chart showing balance trend over the selected period.
R-DASH-9: The system shall allow dashboard data to be exported as a PDF report.
RDASH-10: The system shall update all dashboard components within 2 seconds of date range selection.

3.6 Budget Management and Alert System

3.6.1 Description and Priority

This feature allows users to set monthly budget limits for expense categories and receive automated alerts when spending approaches or exceeds defined thresholds. This feature is classified as High Priority as it directly addresses the problem statement of overspending and lack of budget control identified in the project proposal.

3.6.2 Stimulus/Response Sequences Set

Budget Sequence:
1. User navigates to 'Budgets' section.
2. User selects expense category and enters monthly limit amount.
3. User sets alert thresholds (e.g., warn at 80%, critical at 100%).
4. System validates amount (positive number) and stores budget configuration.
5. System begins tracking spending against the budget from the first day of the month.

Alert Trigger Sequence:
1. User adds an expense transaction.
2. System recalculates category spending for the current month.
3. If spending reaches 80% of budget, system generates warning alert. 4. If spending reaches 100% of budget, system generates critical alert.
5. Alert is displayed in the UI notification center and optionally sent via email.

Budget Report Sequence:
1. User navigates to 'Budget Report' page.
2. System displays all active budgets with progress indicators.
3. System shows historical budget adherence (previous months).
4. System provides recommendations based on spending patterns.

3.6.3 Functional Requirements

R-BUD-1: The system shall allow users to set monthly budget limits for any expense category.
R-BUD-2: The system shall support configurable alert thresholds: Warning (default 80%), Critical (default 100%).
R-BUD-3: The system shall display real-time budget utilization as progress bars in the dashboard.
R-BUD-4: The system shall generate visual alerts (banner notifications, color-coded indicators) when thresholds are reached.
R-BUD-5: The system shall send email notifications for critical budget alerts (optional, user-configurable).
R-BUD-6: The system shall prevent budget overage or display overage amount clearly in red.
R-BUD-7: The system shall allow users to edit or deactivate budgets mid-month.
R-BUD-8: The system shall provide a budget vs. actual spending comparison report.
RBUD-9: The system shall carry over unused budget amounts to the next month (optional setting).
R-BUD-10: The system shall display budget history and trends over previous 6 months.

4. External Interface Requirements

4.1 User Interfaces

The Smart Expense Tracker Web System provides a responsive, intuitive user interface designed for users with varying levels of technical expertise. The UI follows modern web design principles with a clean, minimalist aesthetic suitable for financial applications.

Interface Components:

• Navigation Bar: Persistent top navigation with links to Dashboard, Transactions, Categories, Budgets, Reports, and Settings. Includes user profile dropdown and notification bell.
• Sidebar (Desktop): Collapsible sidebar on desktop viewports showing quick links and summary stats.
• Dashboard Cards: Metric cards displaying total income, expenses, balance, and savings rate with trend indicators.
• Chart Components: Interactive pie charts (expense distribution), bar charts (income vs expense), and line charts (balance trend) using Recharts library.
• Transaction Forms: Modal dialogs for adding/editing transactions with real-time validation feedback.
• Data Tables: Sortable, filterable tables for transaction lists with pagination (10, 25, 50, 100 rows per page).
• Notification Center: Slide-out panel displaying budget alerts, system messages, and reminders.

Responsive Design Breakpoints:
• Mobile: < 768px - Single column layout, hamburger menu, bottom navigation.
• Tablet: 768px - 1024px - Two column layout, collapsible sidebar.
• Desktop: > 1024px - Full layout with sidebar, multi-column dashboard.

Accessibility Requirements:
• All interactive elements shall have minimum touch target size of 44x44px on mobile.
• Color contrast ratios shall meet WCAG 2.1 AA standards (minimum 4.5:1 for normal text).
• All charts shall include alternative text descriptions and data tables for screen readers.
• Keyboard navigation shall be supported for all primary functions.

4.2 Hardware Interfaces

The system operates as a web application and does not require specialized hardware interfaces. However, the following hardware considerations apply:

Client Hardware:
• Standard computing devices: desktops, laptops, tablets, smartphones.
• Input devices: keyboard, mouse, touch screen.
• Display: minimum resolution 320px width (mobile) to 1920px width (desktop).
• Camera: optional, for receipt image capture on mobile devices.
• Storage: minimal local storage required (IndexedDB for offline caching).

Server Hardware:
• Standard server hardware or cloud VPS (Virtual Private Server).

• Network interface: Ethernet or WiFi for internet connectivity.
• Minimum specifications: 2 CPU cores, 4GB RAM, 20GB SSD storage.

4.3 Software Interfaces

The Smart Expense Tracker Web System interfaces with the following software components:

Frontend Software Interfaces:
• React.js v18.x: Core UI framework.
• React Router v6.x: Client-side routing and navigation.
• Axios v1.x: HTTP client for API communication.
• Recharts v2.x: Charting library for data visualization.
• React Hook Form v7.x: Form validation and management.
• Tailwind CSS v3.x: Utility-first CSS framework for styling.
• date-fns v2.x: Date manipulation and formatting.

Backend Software Interfaces:
• Node.js v18.x LTS: Server runtime environment.
• Express.js v4.x: Web application framework.
• Mongoose v7.x: MongoDB object modeling and data validation.
• jsonwebtoken v9.x: JWT generation and verification.
• bcryptjs v2.x: Password hashing.
• cors v2.x: Cross-origin resource sharing middleware.
• dotenv v16.x: Environment variable management.
• nodemailer v6.x: Email notification service (for budget alerts).

Database Interface:
• MongoDB v6.0+: NoSQL database accessed via Mongoose ODM.
• Connection via MongoDB URI with authentication credentials.
• Supports both local instance and MongoDB Atlas cloud service.

External Service Interfaces:
• Email Service: SMTP server or SendGrid API for transactional emails (password reset, budget alerts).
• Cloud Storage: Optional AWS S3 or Cloudinary for receipt image storage (if local storage is insufficient).

4.4 Communications Interfaces

The system uses standard internet communication protocols for all data exchange:

Network Protocols:
• HTTP/1.1 and HTTP/2: Primary protocol for client-server communication.
• HTTPS: Required for all production deployments (TLS 1.2 or higher).
• WebSocket (optional): For real-time notifications (future enhancement).

API Communication:
• RESTful API architecture with JSON data format.
• Standard HTTP methods: GET (retrieve), POST (create), PUT/PATCH (update), DELETE (remove).
• HTTP status codes: 200 (OK), 201 (Created), 400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), 404 (Not Found), 500 (Internal Server Error).
• Request/Response Content-Type: application/json.
• Authentication header: Authorization: Bearer <JWT_TOKEN>.

Email Communication:
• SMTP protocol for sending transactional emails.
• Email formats: HTML with plain text fallback.
• Supported email types: welcome emails, password reset, budget alerts, monthly summaries.

Data Transmission:
• All API communications shall use JSON format.
• Date format: ISO 8601 (YYYY-MM-DD).
• Currency format: Decimal with two precision places (e.g., 1250.50).
• File uploads: multipart/form-data for receipt images.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The following performance requirements ensure the system operates efficiently under expected load:

R-PERF-1: The system shall load the initial dashboard within 3 seconds on a standard broadband connection (10 Mbps).
R-PERF-2: The system shall process and respond to API requests within 500ms for 95% of requests under normal load.
R-PERF-3: The system shall support a minimum of 100 concurrent users without degradation of response times.
R-PERF-4: The system shall handle database queries with up to 10,000 transactions per user with query response times under 1 second.

R-PERF-5: The system shall render charts and visualizations within 2 seconds of data availability.
R-PERF-6: The system shall complete user authentication processes (login, registration) within 2 seconds.
R-PERF-7: The system shall optimize images (receipt uploads) to a maximum of 500KB per image without significant quality loss.
R-PERF-8: The system shall implement database indexing on frequently queried fields (user_id, date, category, type) to ensure query efficiency.
R-PERF-9: The system shall implement client-side caching for dashboard data to reduce redundant API calls.
R-PERF-10: The system shall support pagination for transaction lists to limit data transfer to 100 records per request.

5.2 Safety Requirements

The following safety requirements ensure data integrity and prevent accidental data loss:

R-SAFE-1: The system shall implement soft delete for all transaction records, retaining deleted data for 30 days before permanent removal.
R-SAFE-2: The system shall display confirmation dialogs for all destructive actions (delete, budget reset, account deletion).
R-SAFE-3: The system shall automatically save draft transaction data in local storage to prevent data loss due to browser crashes.
R-SAFE-4: The system shall implement database backup procedures with daily automated backups.
R-SAFE-5: The system shall validate all user inputs to prevent injection attacks and data corruption.
R-SAFE-6: The system shall maintain an audit log of all data modifications with timestamp and user identification.
R-SAFE-7: The system shall prevent concurrent editing conflicts by implementing optimistic locking or last-write-wins strategy with user notification.
R-SAFE-8: The system shall gracefully handle server errors and display user-friendly error messages without exposing system internals.

5.3 Security Requirements

The following security requirements protect user data and system integrity:

R-SEC-1: The system shall implement JWT-based authentication with secure token storage (HttpOnly cookies or secure localStorage).
R-SEC-2: The system shall enforce HTTPS for all production communications to prevent man-in-the-middle attacks.
R-SEC-3: The system shall hash all passwords using bcrypt with a minimum of 10 salt rounds.

R-SEC-4: The system shall implement rate limiting on authentication endpoints (5 attempts per 15 minutes per IP).
R-SEC-5: The system shall validate and sanitize all user inputs to prevent XSS (CrossSite Scripting) and NoSQL injection attacks.
R-SEC-6: The system shall implement CORS (Cross-Origin Resource Sharing) policies restricting API access to authorized domains.
R-SEC-7: The system shall not store sensitive financial information (bank account numbers, credit card details).
R-SEC-8: The system shall implement session timeout after 30 minutes of inactivity, requiring re-authentication.
R-SEC-9: The system shall log all security events (failed logins, password changes, unauthorized access attempts) for audit purposes.
R-SEC-10: The system shall implement Content Security Policy (CSP) headers to mitigate XSS and data injection attacks.
R-SEC-11: The system shall require strong passwords as defined in R-AUTH-2 and enforce periodic password changes (every 180 days).
R-SEC-12: The system shall encrypt sensitive data at rest (database encryption) and in transit (TLS 1.2+).

5.4 Software Quality Attributes

The following quality attributes define the overall characteristics of the system:

Availability:
- The system shall maintain 99% uptime during operational hours (excluding scheduled maintenance).
- Scheduled maintenance shall be performed during off-peak hours with 24-hour advance notification.

Reliability:
- The system shall have a Mean Time Between Failures (MTBF) of at least 720 hours (30 days).
- The system shall recover from failures within 5 minutes with automatic restart mechanisms.

Maintainability:
- The system code shall follow standard coding conventions (ESLint for JavaScript, Prettier for formatting).
- The system shall have modular architecture allowing independent updates to frontend, backend, and database layers.
- The system shall include comprehensive inline documentation and README files.
- The system shall achieve minimum 70% code coverage in unit tests.

Portability:
- The system shall operate on any platform supporting modern web browsers (Windows, macOS, Linux, iOS, Android).
- The system backend shall be deployable on any platform supporting Node.js (Linux, Windows, cloud platforms).
- The system database shall be portable between local MongoDB instances and MongoDB Atlas cloud service.

Usability:
- The system shall be operable by users with basic computer literacy without requiring training.
- The system shall provide contextual help and tooltips for all primary functions.
- The system shall support keyboard navigation and screen reader compatibility.
- The system shall provide feedback within 1 second for all user actions.

Scalability:
- The system architecture shall support horizontal scaling through stateless API design.
- The database design shall support sharding for future user growth beyond 10,000 active users.

Testability:
- The system shall include unit tests for all API endpoints using Jest testing framework.
- The system shall include integration tests for critical user flows (registration, login, transaction CRUD).
- The system shall support automated end-to-end testing using Cypress or Playwright.

Appendix A: Glossary

The following terms, acronyms, and abbreviations are used throughout this SRS document:

1. API (Application Programming Interface)
a. A set of protocols and tools for building software applications.
b. Defines how software components should interact.

2. bcrypt
a. A password-hashing function based on the Blowfish cipher.
b. Used for secure password storage with adaptive salt rounds.

3. CRUD (Create, Read, Update, Delete)
a. The four basic operations of persistent storage.
b. Fundamental to transaction management in the system.

4. CSV (Comma-Separated Values)
a. A plain text format for storing tabular data.
b. Used for bulk import/export of transaction data.

5. JWT (JSON Web Token)
a. A compact, URL-safe means of representing claims between parties.
b. Used for stateless authentication in the system.

6. MERN Stack
a. MongoDB, Express.js, React.js, Node.js technology stack.
b. The full-stack JavaScript framework used for this project.

7. MongoDB
a. A document-oriented NoSQL database program.
b. Uses JSON-like documents with optional schemas.

8. NoSQL
a. Non-relational database management system.
b. Suitable for unstructured and semi-structured data.

9. REST (Representational State Transfer)
a. An architectural style for designing networked applications.
b. Relies on stateless, client-server communication.

10. Responsive Design
    a. Web design approach for optimal viewing across devices.
    b. Uses fluid grids, flexible images, and CSS media queries.

11. SPA (Single Page Application)
    a. Web application that loads a single HTML page.
    b. Dynamically updates content without page reloads.

12. TLS (Transport Layer Security)
    a. Cryptographic protocol for secure network communication.
    b. Successor to SSL (Secure Sockets Layer).

13. UI (User Interface)
    a. The visual elements through which users interact with software.
    b. Includes screens, pages, buttons, icons, and forms.

14. UX (User Experience)
    a. The overall experience of a person using a product.
    b. Encompasses usability, accessibility, and satisfaction.

15. WCAG (Web Content Accessibility Guidelines)
    a. International standard for web accessibility.
    b. Published by the W3C Web Accessibility Initiative.